YSC - Explainable Deep Learning Models for Anomaly Detection

Data Description

The core transactions dataset we use is the direct HACT (Harmonized Approach to Cash Transfers) payment list from UNICEF programme section in Yemen.

Key dataset used for the model:

- **Verification code** (pseudonymized beneficiary ID)
- **Beneficiary name** (for later rapid fuzzy-match check)
- **Amount** (requested amount)
- **Payment activity description** (service description in Arabic)
- **ID card number** (any national ID)
- **Phone number**
- **Payment order** (contains the program code, e.g., PBG payment)
- **Payment cycle** (extracted from the payment order or from the filename)

Role in the Anomaly Detection Framework

- Ground truth of the beneficiary payment distribution, enabling pre-payment validation against the beneficiary list.
- Feeds the "**historical averages**," "**payment counts**," and "**z-score**", etc., used by our amount-spike and frequency-surge detectors.
- Continuously updated after each disbursement, so anomalies can be flagged both before funds are released and after they're reconciled.

Data Quality and Privacy:

- All numeric fields (amount, cycle IDs) are cast to **float/int** with coercion
- Raw PII is never exposed – only pseudonymized IDs in outputs and all data at rest or in transit is encrypted.

Outlier Detection Deep Learning Model

Deep Learning Models	**Description**	**Backbone**
AutoEncoder(AE)	Encode and rebuild data to detect outliers by measuring reconstruction errors.	AutoEncoder
DeepSVDD	Optimize a hypersphere in the feature space to enclose normal data points	Neural Network
AE1SVM	AutoEncoder with One Support Vector Machine (SVM)	AutoEnconder and SVM

AutoEncoder (AE)

Auto Encoder (AE) is a type of neural networks for learning useful data representations in an unsupervised manner. Similar to PCA, AE could be used to detect outlying objects in the data by calculating the reconstruction errors.

Model Parameters

contamination: float in (0., 0.5), used (0.05)

The amount of contamination of the data set,

i.e. the proportion of outliers in the data set.

Used when fitting to define the threshold on the decision function.

preprocessing: bool, optional (default=False)

If True, apply the preprocessing procedure before training models.

We used **StandardScaler** preprocessing to explicitly control and reuse the same

means/standard deviations for both training and evaluation.

lr: float, optional (default=1e-3)

The initial learning rate for the optimizer.

epoch_num: int, optional (default=10)

The number of epochs for training.

Batch size: int, optional (128)

The batch size for training.

optimizer_name: str, optional (default='adam')

The name of the optimizer used to train the model.

random_state: int, optional (default=42)

The random seed for reproducibility.

use_compile: bool, optional (default=False)

Whether to compile the model. If True, the model will be compiled before training.

This is only available for PyTorch version >= 2.0.0. and Python < 3.12.

compile_mode: str, optional (default='default')

The mode to compile the model.

Can be either “default”, “reduce-overhead”,

“max-autotune” or “max-autotune-no-cudagraphs”.

verbose: int, optional (default=1)

Verbosity mode – (0 = silent, 1 = progress bar, 2 = one line per epoch)

optimizer_params: dict, optional (default={'weight_decay': 1e-5})

Additional parameters for the optimizer.

For example, `optimizer_params={'weight_decay': 1e-5}`.

hidden_neuron_list: list, optional (default=[64, 32])

The number of neurons per hidden layers.

So, the network has the structure as [feature_size, 64, 32, 32, 64, feature_size].

hidden_activation_name: str, optional (default='ReLU')

The activation function used in hidden layers (Rectified Linear Unit).

batch_norm: boolean, optional (default=True)

Whether to apply Batch Normalization: Batch Normalization normalizes layer outputs to have zero mean and unit variance per batch during training, improving stability and convergence in neural networks.

dropout_rate: float in (0., 1), optional (default=0.2)

The dropout to be used across all layers. dropout is a form of regularization in AutoEncoders.

1. Overall Architecture

The ***AutoEncoder*** inherits from ***BaseDeepLearningDetector*** and builds a symmetric **5-layer network**:

Input (d=4) 4-dimensional input features

↓

$W_1 \in R^{4x64}$, $b_1 \in R^{64}$

Linear → ReLU → BN ⟶ Hidden 1 (64 dims)

↓

$W_2 \in R^{64x32}$, $b_2 \in R^{32}$

Linear → ReLU → BN ⟶ Code Layer (32 dims)

↓

$W_3 \in R^{32x64}$, $b_3 \in R^{64}$

Linear → ReLU → BN ⟶ Hidden 2 (64 dims)

↓

$W_4 \in R^{64x4}$, $b_4 \in R^{4}$

Linear ⟶ Reconstruction (4 dims)

The encoder is the first two Linear + ReLU + BN layers, mapping 4d→64 dims→32 dims.

The decoder mirrors that: 32 dims→64 dims→4d.

In our AutoEncoder, each weight matrix $\boldsymbol{W_i}$ and the bias vector $\boldsymbol{b_i}$ is initialized automatically, seeded by **random_state** to ensure reproducibility. So, we never hand-pick those values. During training, **gradient descent** optimization (Adam) then adjusts every $\boldsymbol{W_i}$ and $\boldsymbol{b_i}$ to minimize the average reconstruction error, effectively finding the best parameters (best optimum) to capture normal transaction patterns.

2. Layer Learned Weights and Biases

Note: All the parameters computed in this model is attached file as **ae_weights**

In practice, the only manual choices we make are the high-level hyperparameters like (**hidden_neuron_list** = [64,32]), **learning rate**, **batch size**, and **number of epochs**, and the training process learns all the individual weights and biases by itself.

3. Layer-by-Layer Equation

1. Input Layer
   Let $\in R^4$ be transaction vector which is the first layer
   Transaction vector: [amount, z-sore, count of past payment per cycle, cycle since last payment]
   - **Magnitude of payment**: (Amount)
   - **Deviation from past payment**: (z-score)
   - **Payment frequency**: (Count of payment per beneficiary per cycle)
   - **Timing irregularities**: (Cycle since last payment)

2. Hidden Layer 1 (64 units)

$$h^{(1)} = Dropout\ (ReLU\ \left(BN\ (W_1 x +\ b_1)\right))$$

   - $W_1 \epsilon R^{4\times 64}$, $b_1 \epsilon R^{64}$ learned
   - A learned weight matrix that linearly combines your 4 inputs into 64 features.

3. Code Layer (32 units)

$$h^{(2)} = Dropout\ (ReLU\ \left(BN\ \left(W_2 h^{(1)} +\ b_2\right)\right)$$

   - $W_2 \epsilon R^{64\times 32}$, $b_2 \epsilon R^{32}$
   - Learns to compress the 64-dim hidden vector into a 32-dim code.

4. Hidden Layer 2

$$h^{(3)} = Dropout\ (ReLU\ \left(BN\ \left(W_3 h^{(2)} +\ b_3\right)\right)$$

- $W_3 \epsilon R^{32\times64}$, $b_3 \epsilon R^{64}$
- Learns to expand the 32-dim code back up toward the original dimensionality (Hidden Layer 1)

5. Reconstruction (Output) Layer

$$\hat{x} =\ W_4 h^{(3)} +\ b^4$$

- $W_4 \epsilon R^{64\times4}$, $b_4 \epsilon R^4$
- A final linear map that rebuilds a 4-dimensional vector x
- No activation here: we want the raw real-valued reconstruction.

W is the learned weight

ReLU(u) = max (0, u) is an element-wise **nonlinearity** that outputs the input directly if it's positive and outputs zero if it's negative or zero.

Dropout (p=0.2) is applied after every activation.

BatchNorm (BN) stabilizes training

Activation = ReLU throughout (no final activation on the **output layer** as we want to maintain real-valued reconstruction).

All $\boldsymbol{W_i},\ \boldsymbol{b_i}$ are initialized randomly (**seed=42**) and optimized via Adam (**lr=1e-3**, **weight_decay=1e-5**) to **minimize** the mean-squared error:

$$L(\theta) = \frac{1}{N}\sum_{I=1}^{N} ||xi - \widehat{x}i\ ||_2^2$$

Where **N** is the number of training examples.

4. The Reconstruction Equation

Feeding a transaction into the trained network $x \epsilon R^4$

$$\hat{x} = W_4\sigma(W_3\sigma(W_2\sigma(W_1 x + b_1) + b_2) + b_3) + b_4$$

A transaction $x \epsilon R^4$ is normal if its squared error

$$\delta(x) = \sum_{j=1}^{4} (xi + \hat{x}j)^2 \leq \tau$$

Where input x is a 4-element column [amount, z-sore, count of past payment per cycle, cycle since last payment]

Flag as anomaly if $\delta(x) > \tau$

τ is the **99th percentile** of those training errors because we assumed about 1% of all data could be anomalies, so anything that reconstructs worse than 99% of the **normal** set is suspect.

If $\delta(x)$ is **small**, the model rebuilt x accurately → Normal data it learned

If $\delta(x)$ is **large**, the model struggled → x is unusual under the learned patterns

5. AE on the normal data (Transactions)

Mean squared reconstruction error per feature (AE-normal) - $\delta(x)$

Feature	**Mean Reconstruction error per feature**
Timing irregularities: (**Cycle since last payment**)	0.017640
Magnitude of payment: (**Amount**)	0.012036
Payment frequency: (Count of payment per beneficiary per cycle)	0.009683
Deviation from past payment: (**z-score**)	0.001286

- The **Cycle since last payment** has by far the highest mean reconstruction error (0.017640), meaning the AE finds payment cycle timing irregularities the hardest to model, so anomalies there may be partially masked by this noisy baseline.

- **Amount** comes next (0.012036), showing the model captures magnitude of payment better than timing irregularity, but still with substantial variability, making payment amount-based anomalies moderately detectable.

- **Count of payment per beneficiary per cycle** has a lower error (0.009683), indicating the AE learns payment frequency more consistently, so unusual high payment frequency are more likely to trigger a clear anomaly signal.

- The **z-score deviation** exhibits the lowest error (0.001286), reflecting a very tight baseline on personalized deviations; hence any transaction that departs from its own historical pattern will stand out most sharply.

DeepSVDD Model

Deep One-Class Classifier is a type of neural network for learning useful data representations in an unsupervised way. DeepSVDD trains a neural network while minimizing the volume of a hypersphere that encloses the network representations of the data, forcing the network to extract the common factors of variation. Similar to PCA, DeepSVDD could be used to detect outlying objects in the data by calculating the distance from center.

Model Parameters

n_features: int,

Number of features in the input data

c: float, optional (default='forwad_nn_pass')

Deep SVDD center, the default will be calculated based on network initialization first forward pass. To get repeated results, we set *random_state*.

use_ae: bool, optional (default=False)

The AutoEncoder type of DeepSVDD it reverse neurons from hidden_neurons if set to True.

hidden_neurons: list, optional (default=[64, 32])

The number of neurons per hidden layers. if use_ae is True, neurons will be reversed eg. from [64, 32] to [64, 32, 32, 64, n_features] like the AutoEncoder model

hidden_activation: str, optional (default='relu')

Activation function to use for hidden layers. All hidden layers are forced to use the same type of activation

output_activation: str, optional (default='sigmoid')

Activation function to use for output layer

optimizer: str, optional (default='adam')

String (name of optimizer) or optimizer instance

epochs: int, optional (10)

Number of epochs to train the model.

batch_size: int, optional (128)

Number of samples per gradient update.

dropout_rate: float in (0., 1), optional (default=0.2)

The dropout to be used across all layers.

l2_regularizer: float in (0., 1), optional (default=0.1)

controls how strongly the model penalizes large weights in each layer to reduce overfitting, using L2 regularization by default.

validation_size: float in (0., 1), optional (default=0.1)

The percentage of data to be used for validation

Preprocessing: bool, optional (default=True)

If True, apply standardization on the data

We leave it as True, because DeepSVDD has already fitted its own scaler internally as svdd.scaler for evaluation.

random_state: int, optional (default=42)

The random seed for reproducibility.

contamination: float in (0., 0.5), used (0.05)

The amount of contamination of the data set, i.e. the proportion of outliers in the data set. When fitting this is used to define the threshold on the decision function.

1. Network Architecture & Learned Parameters

By default (use_ae=False, hidden_neurons = [64,32]), DeepSVDD builds this simple 2-layer network:

Input $x \epsilon R^d$	
↓	
Linear (d → 64, bias = False)	→ $W_1 \epsilon R^{d \times 64}$
↓ ReLU (hidden_activation = 'relu')	
↓	
Linear (64 → 32, bias = False)	→ $W_2 \epsilon R^{64 \times 32}$
↓ ReLU	
↓	
Output $z \epsilon R^{32}$	

All weights are initialized (via the PyTorch default) and then learned by gradient descent.

2. Hypersphere Center C

DeepSVDD compute (c) is the **mean** of $z \epsilon R^{32}$ averaging column-wise: written in python as ***self.c = torch.mean(z, dim=0)***. If c is not manually set, the model will compute it automatically during the **first forward pass** on the training data. So, c is initialized as the mean of all encoded vector z_i from the first pass (plus a small ε adjustment to avoid 0s)

$$c = \frac{1}{N}\sum_{i=1}^{N} z_i \, \varepsilon \, R^{32}$$

c is the **center** of the "normal" codes in latent space.

3. Defining Training Objective

On each batch of size B **(batch_size** = 128), the model:

I. Encodes $x_i \rightarrow z_i = \emptyset_\theta(x_i)$
   - x_i: is the original d-dimensional transaction
   - $\emptyset_\theta$: the learned map (two linear layers + ReLU) parameterized by weights $\{W_1 , W_2\}$.

- z_i : the compressed representation of x_i from the output $z \epsilon R^{32}$

II. Computes squared distance

$$D_i = || z_i - c ||_2^2 = \sum_{k=1}^{32} (z_i k - c_k)^2$$

III. Loss

$$\ell_{batch} = \underbrace{\frac{1}{B} \sum_{i=1}^{B} D_i}_{\downarrow} + \underbrace{\lambda \sum_{l} ||W_l||_2}_{\downarrow}$$

*1. The first term squeezes codes **toward c*** *2. Penalize large weights (L2 reg)*

Gradient descent (Adam) adjust $\boldsymbol{W_l}$, where represent $(\boldsymbol{W_1}, \boldsymbol{W_2})$**,** to minimize this loss over the 10 epochs.

4. Decision Function & Threshold

After training

1. Score any new transaction x by
   $z = \emptyset_\theta(x), \quad score(x) = D(x) = \sum_{k=1}^{32} (z_k - c_k)^2$

2. **Threshold** $\tau = the\ (1 - \boldsymbol{contamination})$: quantile of all training $score(x)$
3. A transaction is **normal if**

$$\sum_{k=1}^{32} (z_k - c_k)^2 \leq \tau$$

5. Pattern

- The two weight matrics $\boldsymbol{W_1} \epsilon R^{d \times 64}$, $\boldsymbol{W_2} \epsilon R^{64 \times 32}$ together learn a nonlinear map $\emptyset_\theta : x \rightarrow z$ that contracts the bulk of our transactions data into a tight 32-dimensional cluster around c.
- **Normal transactions** are those whose encoded representation lie inside that learned hypersphere
- Anything that land outside, i.e., whose square distance to c exceeds τ is flagged as an anomaly.

6. Full Equation Form

A transaction of $x \in R^d$ is norma if the following holds:

$$\underbrace{\sum_{k=1}^{32} [\sigma(W_2\sigma(W_1x))_k - c_k]^2}_{D(x)} \leq \tau$$

Where:

- σ = ReLU
- $\boldsymbol{W_1} \in R^{d\times 64}$, $\boldsymbol{W_2} \in R^{64\times 32}$ are the learned weights
- $c \in R^{32}$ is the learned center
- τ is the 99th percentile cutoff of $D(x)$ on training data

7. Layer Learned Weights and Biases

Note: All the parameters computed in the DeepSVDD model is attached as **svdd_weights**

8. SVDD on the normal data (Transaction)

Using the input gradient saliency:

- We treat SVDD distance $\sum_k (z_k - c_k)^2$ as a loss and backpropagate into the inputs.
- The average absolute gradient $\left| \partial distance / \partial_{x_j} \right|$ for feature j tells us how sensitively the distance is, and thus anomalous.

Input-Gradient Saliency for each feature for the normal data (transaction).

Feature	**Average Input Gradient**
Payment frequency: (Count of payment per beneficiary per cycle)	0.149287
Timing irregularities: (**Cycle since last payment**)	0.139634
Magnitude of payment: (**Amount**)	0.114174
Deviation from past payment: (**z-score**)	0.078548

- The **Count of payment per beneficiary per cycle** shows the highest saliency (~0.149). DeepSVDD's normal-data boundary is shaped most sharply by each user's

payment frequency, meaning, small changes there move you farthest from the learned hypersphere.

- **Cycles since last payment** follows closely with frequency, with a saliency of (~0.1396), indicating that uneven gaps between payments do influence the boundary to a great extent.
- **Payment magnitude** comes next (~0.114), so while frequency and time irregularity are primary, the model still treats transaction size as a key dimension of "normal" behavior, meaning, large swings in amount would strongly affect the distance.

- **Z-score deviation** has the lowest saliency (~0.0785), implying personalized deviation from one's own history plays the smallest direct role in defining the normal-data sphere - though it still contributes, it's the gentlest lever on the distance metric.

AE-1SVM

AE-1SVM combines the strengths of AutoEncoders and **One-Class SVM** for unsupervised anomaly detection. The AutoEncoder first learns a compressed and denoised representation of the input data, capturing key patterns in the normal behavior. This latent space is then used by the One-Class SVM to define a tight decision boundary around the normal instances. In practice they work as standalone unsupervised anomaly detection models. However, by separating feature learning (**via AE**) from anomaly boundary detection (**via 1SVM**), the model achieves more robust and accurate detection, especially in high-dimensional or noisy datasets.

Model Parameters

hidden_neurons: list, optional (default=[64, 32])

Number of neurons in each hidden layer.

hidden_activation: str, optional (default='relu')

Activation function for the hidden layers.

batch_norm: bool, optional (default=True)

Whether to use batch normalization.

learning_rate: float, optional (default=1e-3)

Learning rate for training the model.

epochs: int, optional (10)

Number of training epochs.

batch_size: int, optional (128)

Size of each training batch.

dropout_rate: float, optional (default=0.2)

Dropout rate for regularization.

weight_decay: float, optional (default=1e-5)

Weight decay (L2 penalty) for the optimizer.

preprocessing: bool, optional (False)

Whether to apply standard scaling to the input data.

We used **StandardScaler** preprocessing to explicitly control and reuse the same means/standard deviations for both training and evaluation.

loss_fn: callable, optional (default=torch.nn.MSELoss)

Loss function to use for reconstruction loss.

contamination: float, used (0.05)

Proportion of outliers in the data.

alpha: float, optional (default=1.0)

Weight for the reconstruction loss in the final loss computation.

sigma: float, optional (default=1.0)

Scaling factor for the random Fourier features.

kernel_approx_features: int, optional (default=1000)

Number of random Fourier features to approximate the kernel.

1. Overall Architecture

AE-1SVM mixes a classic AutoEncoder with a One-Class SVM on random Fourier features. It has three main parts

1. **Encoder**:
   - Layer sizes
     $d \rightarrow 64 \rightarrow 32$
     Where d is our input dimension (4 features)
   - Each layer i does:
     $$h^{(i)} = Dropout\ (ReLU\ \left(BN\ \left(W_i h^{(i-1)} + b_i\right)\right)$$
     With

     $W_1 \epsilon R^{d\times 64}$, $b_1 \epsilon R^{64}$, $\quad W_2 \epsilon R^{64\times 32}$, $b_2 \epsilon R^{32}$
   - The result is a 32-dim code z

2. **Random Fourier Features (RFF)**:
   - Transform $z = R^{32}$ into a high-dimensional feature $\emptyset(z) \epsilon R^D$ with
     $$\emptyset(z) = \cos(z\, W_{rff} + b_{rff})$$
     where $\quad W_{rff} \epsilon R^{32\times D}\ b_{rff} \epsilon R^D$

   $D = kernel_aprox_features\ \ (default\ 1000)$, and cos is applied element-wise.

3. **Decoder**: Reconstruct back to x from **step 1**, output will be used to calculate $\boldsymbol{L_{rec}}$
   - Mirrors the encoder sizes in reverse:

     $32 \rightarrow 64 \rightarrow d$

     With parameters

     $\{W_3 \epsilon R^{32\times 64}$, $b_3 \epsilon R^{64}$; $\quad W_4 \epsilon R^{64\times d}$, $b_4 \epsilon R^d\}$
   - This produces the reconstruction $\hat{x}$

4. **One-Class SVM head**: Using the result of our RFF high-dim feature $\emptyset(z)$ in **step 2**
   - A single linear layer on $\emptyset(z)$: This train a linear model
     $$f_{svm}(z) = w_{svm}\emptyset(z) + b_{svm}$$
     where
     $w_{svm} \epsilon R^D$, $b_{svm} \epsilon R$

All weights are initialized (via the PyTorch default) and then learned by gradient descent. To get a reproducible set of initial weights and biases, we call $\boldsymbol{torch.manal_seed(42)}$

2. Training Objective

On each batch of B (128) samples $\{x_i\}$:

1. **Reconstruction loss**

   $$L_{rec} = \frac{1}{B}\sum_{i=1}^{B} ||x_i - \hat{x}_i||_2^2 = \frac{1}{B}\sum_{i=1}^{B}\sum_{j=1}^{d}(x_{i,j} - \hat{x}_{i,j})^2$$

   The L_{rec} is used as **loss_fn**, which we have set as the Loss function for reconstruction loss.

2. **SVM hinge loss**

   $$L_{svm} = \frac{1}{B}\sum_{i=1}^{B} max\ \{0, 1 - f_{svm}\ (z_i)\}$$

   Points for which $f_{svm}\ (z_i)\ \geq 1$ lie safely on the correct side of the boundary; any point with $f_{svm}\ (z_i)\ < 1$ violates the margin and incurs a penalty proportional to how far inside the margin it falls. The hinge loss L_{svm} therefore measures and penalizes exactly those margin violations, encouraging the model during training to push all normal codes out to $f_{svm}\ (z_i)\ \geq 1$.

3. **Combined loss**: Full equation for the training with RFF + SVM acting as regularizer and L2 penalty (L2 regularization)

   $$L_{batch} = \alpha\, L_{rec} + L_{svm} + \lambda \sum_{l=1}^{4} ||W_l||_2^2$$

   - Where α ($\boldsymbol{alpha}$) balances reconstruction vs SVM (default as 1.0), and λ is the **weight_decay.**
   - Setting $\alpha = 1$ mean we're treating reconstruction and hinge loss on equal footing. If we want to de-emphasize the SVM part, we could raise α above 1; if we want to make the SVM head more dominant, we'd lower α below 1
   - **Optimization**: Adam updates all $\{W_i,\ b_i\}$, plus $\{W_{rff},\ b_{rff}, w_{svm},\ b_{svn}\}$ over **10 epochs**.

3. Layer Learned Weights and Biases

Note: All the parameters computed in the AE1SVM model is attached below as **ae1svm_weights**

4. Scoring and Normal Transactions

After training, AE-1SVM discards the SVM head at inference and scores each x by it reconstruction error: scores new samples

$$\delta(x) = ||x - \hat{x}||_2^2 = \sum_{j=1}^{d} (x_j - \hat{x}_j)^2$$

A normal transaction satisfies

$$\delta(x) \leq \tau ,$$

Where τ is the 99th percentile of $\{\delta(x_i)\}$ on the training data given $\boldsymbol{contamination = 0.01}$

5. Full Learned Parameter for Normal Transaction

- **Encoder weights** W_1, b_1, W_2, b_2: define the 2-stage nonlinear code z
- **Decoder weights** W_3, b_3, W_4, b_4: reconstruct that code back to x.
- **Random Fourier Features** parameters W_{rff}, b_{rff} and SVM head w_{svm}, b_{svn}: shape the latent space during training, forcing codes to be easily separable from the 'fake' anomalies.
- **Threshold $\boldsymbol{\tau}$**: learned cutoff on the reconstruction error.

6. AE1SVM on the normal data (Transaction)

Mean squared reconstruction error per feature (AE1SVM-normal)

Feature	**Mean Reconstruction error per feature**
Timing irregularities: (**Cycle since last payment**)	0.267763
Magnitude of payment: (**Amount**)	0.165379
Payment frequency: (Count of payment per beneficiary per cycle)	0.156347
Deviation from past payment: (**z-score**)	0.000090

The result of the AE1SVM is similar to the AutoEncoder model, but more optimized for certain features.

On the AE-1SVM "normal" set, **cycles since last payment** now carry the largest mean error (0.2678), nearly 15 × AE's 0.0176, showing that adding the SVM head makes timing irregularities much noisier to reconstruct. **Payment amount** and **payment frequency** errors also swell (0.1654 - AE1SVM vs. 0.0120 - AE, for amount, 0.1563 - AE1SVM vs. 0.0097 - AE for frequency), indicating the joint AE+SVM struggles more with these global features. In contrast, the **z-score** error collapses to 0.00009 (down from 0.0013), meaning personalized deviations are modeled almost perfectly and will jump out sharply when abnormal. Compared to the standalone AE, AE-1SVM yields a much tighter baseline on z-score (boosting sensitivity there) but a noisier baseline on raw amounts and timing, shifting the model's anomaly focus toward personalized deviation.

Extreme Outlier Outputs

The top 100 anomalies from each **AutoEncoder**, **AE1SVM**, and **DeepSVDD** models are attached below in an excel file as **Top_100_DL model_with_errors**.

The output includes the **original input features**, **fake record identifiers**, **per-feature reconstruction errors** (or **Input–Gradient Saliency scores** for DeepSVDD) and the **overall average error**. Anomaly flags are determined by combining signals from all four features according to their respective error magnitudes or gradient sensitivities.

- **Payment Amount**
  As the core transactional metric, anomalies here typically involve overpayments, underpayments or unexpected values. A high reconstruction error in this feature indicates an amount that deviates significantly from what the model has learned as "normal."

- **Z-Score**
  This feature measures how far the current payment departs from a beneficiary's personal history. It is especially powerful for catching personalized anomalies whereby small absolute deviations can stand out sharply when they fall outside an individual's typical range.

- **Count of Payment per Beneficiary Per Cycle**:
  This feature captures how many times a beneficiary appears in a single payment cycle. It helps identify duplication or unexpected multiple payments. Depending on the payment cycle, the model should be able to flag potential duplicate records, double

disbursements, or irregular entries such as a beneficiary receiving more than 1 to 3 payments within the same cycle or being listed multiple times.

- **Cycle since last payment cycle**:
  This feature captures frequency consistency. Unexpected gaps or shortened cycles (e.g., someone getting paid again within 1 or 2 cycles, can signal anomalies and suggest irregular or unauthorized payments)

Often, anomalies arise from combinations of these signals, such as an unusual amount coupled with a high z-score, or a spike in count alongside a shortened cycle interval. While the model can occasionally misclassify genuine transactions, those false positives are typically identified and excluded during review.